# Automação com Python para Iniciantes

## Como Criar Robôs que Trabalham por Você

**Automatize tarefas repetitivas e economize horas de trabalho usando Python.**

## Sumário


1. O que é automação
2. Por que Python é perfeito para automação
3. Instalando Python
4. Seu primeiro script automático
5. Automatizando o mouse e teclado (PyAutoGUI)
6. Automatizando tarefas no computador
7. Automatizando a web (Selenium)
8. Automatizando planilhas (Pandas)
9. Criando bots simples (juntando tudo)
10. Ideias para ganhar dinheiro com automação
11. Capítulo bônus: 10 ideias de projetos
12. Bônus detalhados (1 a 20)
13. Bônus Extra: Robô de Produtividade Completo


## Como usar este eBook

- Leia na ordem: cada capítulo prepara o próximo.
- Copie os códigos e execute: você aprende mais “fazendo” do que só lendo.

- Não se preocupe em decorar: entenda a ideia e repita os exercícios.

**Aviso importante**: automação serve para facilitar seu trabalho, não para burlar regras. Em sites, respeite termos de uso, limites e privacidade. Evite automatizar ações em contas de terceiros sem permissão.

## Pré-requisitos (bem básicos)

- Um computador com Windows, macOS ou Linux
- Internet para instalar ferramentas e bibliotecas
- Vontade de testar e errar (isso faz parte)

---

# 1) O que é automação

## 1.1 Introdução do capítulo

Automação é quando você faz o computador executar tarefas repetitivas por você. Em vez de clicar 200 vezes por dia, você cria um “robô” (um programa) que faz isso rápido e do mesmo jeito sempre.

## 1.2 Explicação teórica simples

Automação normalmente envolve:

- **Entrada**: de onde vem a informação (arquivo, site, planilha, e-mail, etc.)
- **Regras**: o que deve ser feito (passo a passo)
- **Saída**: o resultado (um arquivo gerado, um formulário preenchido, uma mensagem enviada, etc.)

### Exemplos reais do dia a dia

- Renomear centenas de arquivos de uma pasta

- Baixar relatórios de um site e salvar em uma planilha
- Preencher dados no sistema da empresa a partir de um Excel
- Enviar e-mails padronizados para uma lista
- Gerar PDFs automaticamente

## 1.3 Exemplo de código (conceito)

Vamos fazer uma automação “de verdade” em miniatura: criar automaticamente uma pasta e um arquivo de texto dentro dela.

```
from pathlib import Path

pasta = Path("meus_relatorios")
pasta.mkdir(exist_ok=True)

arquivo = pasta / "relatorio.txt"
arquivo.write_text("Relatório gerado automaticamente!\n", encoding="utf-8")

print("Pronto! Pasta e arquivo criados.")
```

## 1.4 Explicação do código (linha por linha)

- `from pathlib import Path`

  Importa uma ferramenta do Python para lidar com pastas/arquivos de forma simples.

- `pasta = Path("meus_relatorios")`

  Cria um “caminho” apontando para uma pasta chamada `meus_relatorios`.

- `pasta.mkdir(exist_ok=True)`

  Cria a pasta. Se ela já existir, não dá erro ( `exist_ok=True` ).

- `arquivo = pasta / "relatorio.txt"`

Cria o caminho do arquivo dentro da pasta.

- `arquivo.write_text(...)`

  Cria o arquivo e escreve o texto dentro dele (usando UTF-8).

- `print(...)`

  Mostra uma mensagem no terminal.

## 1.5 Exercício prático

Crie uma pasta chamada `backup` e dentro dela crie um arquivo `data.txt` com o texto: “Backup criado”.

## 1.6 Desafio extra

Em vez de escrever um texto fixo, escreva a data e hora atual no arquivo. (Dica: módulo `datetime`.)

---

# 2) Por que Python é perfeito para automação

## 2.1 Introdução do capítulo

Python é uma das linguagens mais usadas no mundo para automação porque é fácil de ler e tem bibliotecas prontas para quase tudo.

## 2.2 Explicação teórica simples

Por que Python é tão bom para automação:

- **Sintaxe simples**: parece “quase português”
- **Bibliotecas**: PyAutoGUI (mouse/teclado), Selenium (web), Pandas (planilhas), Requests (internet), etc.
- **Comunidade enorme**: você encontra exemplos e soluções com facilidade
- **Roda em qualquer sistema**: Windows, macOS, Linux

### O que é uma biblioteca?

É um “pacote” de código pronto que alguém já escreveu para facilitar sua vida. Você instala e usa.

## 2.3 Exemplo de código (por que “ler” é fácil)

```
nome = input("Qual é seu nome? ")
print("Olá,", nome, "! Python é bem amigável.")
```

## 2.4 Explicação do código

- `input(...)` pede algo para a pessoa digitar.
- O texto digitado vai para a variável `nome`.
- `print(...)` mostra a mensagem.

## 2.5 Exercício prático

Faça um programa que peça o nome e a idade e depois mostre uma frase: “Você tem X anos”.

## 2.6 Desafio extra

Peça dois números e mostre a soma.

---

# 3) Instalando Python

## 3.1 Introdução do capítulo

Antes de automatizar qualquer coisa, precisamos instalar o Python e preparar um ambiente simples para rodar nossos scripts.

## 3.2 Explicação teórica simples

Você vai usar:

- **Python 3.x**
- **Terminal** (Prompt de Comando, PowerShell ou Terminal do macOS/Linux)
- **Um editor** (recomendado: VS Code)

### Verificando se já tem Python

No terminal, digite:

- Windows: `python --version`
- macOS/Linux: `python3 --version`

## 3.3 Exemplo de "código" (comandos essenciais)

Instalar bibliotecas (pacotes) com o **pip**:

```
pip install pyautogui
pip install selenium
pip install pandas
```

## 3.4 Explicação (comandos)

- `pip` é o instalador de pacotes do Python.
- `pip install nome` baixa e instala a biblioteca.
- Se der erro de permissão, tente:
  - Windows: abrir terminal como administrador
  - macOS/Linux: `python3 -m pip install ...` ou usar ambiente virtual

### (Recomendado) Criando um ambiente virtual

Ambiente virtual isola suas bibliotecas por projeto.

```
python -m venv .venv
```

Ativar:

- Windows (PowerShell):

```
.\.venv\Scripts\Activate.ps1
```

- macOS/Linux:

```
source .venv/bin/activate
```

## 3.5 Exercício prático

1. Instale o VS Code.
2. Verifique a versão do Python no terminal.
3. Crie uma pasta de projeto e um ambiente virtual `.venv`.

## 3.6 Desafio extra

Instale a biblioteca `pandas` dentro do ambiente virtual e confirme com:

```
pip show pandas
```

---

# 4) Seu primeiro script automático

## 4.1 Introdução do capítulo

Agora vamos criar um script que faz uma tarefa repetitiva simples: organizar arquivos por extensão.

## 4.2 Explicação teórica simples

Muita automação começa com:

1. Ler algo (arquivos de uma pasta)
2. Tomar uma decisão (se é `.pdf`, vai para "PDF")
3. Mover/renomear/criar coisas

## 4.3 Exemplo de código (organizar downloads)

> *Este exemplo move arquivos de uma pasta para subpastas. Teste primeiro em uma pasta de testes.*

```python
from pathlib import Path
import shutil

pasta_origem = Path("pasta_teste")
pasta_origem.mkdir(exist_ok=True)

for arquivo in pasta_origem.iterdir():
    if arquivo.is_file():
        extensao = arquivo.suffix.lower().replace(".", "")
        if extensao == "":
            extensao = "sem_extensao"

        destino = pasta_origem / extensao
        destino.mkdir(exist_ok=True)

        shutil.move(str(arquivo), str(destino / arquivo.name))

print("Arquivos organizados!")
```

## 4.4 Explicação do código

- `Path("pasta_teste")` aponta para a pasta.
- `mkdir(exist_ok=True)` cria a pasta se não existir.
- `iterdir()` lista tudo dentro da pasta.
- `is_file()` garante que é arquivo (não pasta).
- `suffix` pega a extensão (ex: `.pdf`).
- `destino.mkdir(...)` cria a subpasta da extensão.
- `shutil.move(...)` move o arquivo para a subpasta.

## 4.5 Exercício prático

Crie 5 arquivos de teste (ex: `a.txt`, `b.txt`, `c.pdf`, `d.jpg`, `e`) e rode o script para organizar.

## 4.6 Desafio extra

Em vez de organizar por extensão, organize por data (crie subpastas `2026-04`, etc.). (Dica: `arquivo.stat().st_mtime`)

---

# 5) Automatizando o mouse e teclado (PyAutoGUI)

## 5.1 Introdução do capítulo

Agora vamos automatizar ações "como se você estivesse usando o computador": mover o mouse, clicar e digitar.

## 5.2 Explicação teórica simples

A biblioteca **PyAutoGUI** controla:

- Mouse: mover, clicar, arrastar
- Teclado: digitar, pressionar teclas
- Tela: localizar imagens (ex: achar um botão pelo print)

### Atenção

Automação de mouse/teclado é sensível:

- Se você mexer no mouse enquanto roda, pode atrapalhar.
- Mudanças de resolução/tema do sistema podem quebrar a busca por imagem.
- Sempre teste com calma.

## 5.3 Exemplo de código (abrir o Bloco de Notas e digitar – Windows)

> *Ajuste conforme seu sistema. Em macOS/Linux, você pode abrir apps de outra forma.*

```
import pyautogui
import time

pyautogui.PAUSE = 0.5  # pausa automática entre comandos
pyautogui.FAILSAFE = True  # mover mouse para o canto superior esquerdo para
parar

# Espera 3 segundos para você preparar a tela
time.sleep(3)

# Abre o menu iniciar (tecla Windows)
pyautogui.press("win")

# Digita "bloco de notas" e aperta Enter
pyautogui.write("bloco de notas", interval=0.05)
pyautogui.press("enter")

time.sleep(1)

# Digita um texto
pyautogui.write("Olá! Isso foi digitado automaticamente pelo Python.",
interval=0.03)
pyautogui.press("enter")
pyautogui.write("Automação é incrível!", interval=0.03)
```

## 5.4 Explicação do código

- `import pyautogui` importa a biblioteca.
- `PAUSE` adiciona uma pausa para ficar mais “humano” e seguro.
- `FAILSAFE` permite parar o script: leve o mouse para o canto superior esquerdo.
- `time.sleep(3)` te dá tempo para não ser pego de surpresa.

- `press("win")` aperta a tecla Windows.
- `write(..., interval=...)` digita com intervalo entre letras.
- `press("enter")` confirma.

## 5.5 Exercício prático

Crie um script que:

1. Abra o menu iniciar
2. Abra o Bloco de Notas
3. Digite seu nome em 3 linhas diferentes

## 5.6 Desafio extra

Use `pyautogui.position()` e `pyautogui.size()` para imprimir no terminal:

- a posição atual do mouse
- o tamanho da tela

---

# 6) Automatizando tarefas no computador (arquivos, pastas, relatórios)

## 6.1 Introdução do capítulo

Automação não é só “clicar”. Muitas tarefas podem ser automatizadas diretamente com arquivos, pastas e dados, sem precisar simular o mouse e teclado. Isso é mais rápido e menos propenso a erros.

## 6.2 Explicação teórica simples

As bibliotecas `pathlib` e `shutil` do Python são excelentes para:

- **Criar/Excluir pastas**
- **Mover/Copiar arquivos**
- **Renomear arquivos**
- **Listar conteúdo de pastas**
- **Verificar informações de arquivos** (tamanho, data de modificação)

### Quando usar?

- Organizar downloads
- Fazer backup de documentos
- Processar lotes de arquivos (ex: converter, compactar)
- Gerar relatórios a partir de dados em arquivos

## 6.3 Exemplo de código (copiar arquivos)

```python
from pathlib import Path
import shutil

pasta_origem = Path("documentos_origem")
pasta_destino = Path("documentos_backup")

pasta_origem.mkdir(exist_ok=True)
pasta_destino.mkdir(exist_ok=True)

# Cria um arquivo de teste na origem
(pasta_origem / "relatorio_vendas.txt").write_text("Conteúdo do relatório")

shutil.copy(str(pasta_origem / "relatorio_vendas.txt"), str(pasta_destino /
"relatorio_vendas_copia.txt"))

print("Arquivo copiado!")
```

## 6.4 Explicação do código

- `Path(...)` define os caminhos.
- `mkdir(...)` cria as pastas.
- `shutil.copy(...)` copia o arquivo de um lugar para outro.

## 6.5 Exercício prático

Crie uma pasta `fotos_ferias` com algumas imagens e um script que copie todas para uma pasta `backup_fotos`.

## 6.6 Desafio extra

Em vez de copiar, mova os arquivos e depois apague a pasta original (dica: `shutil.rmtree`).

---

# 7) Automatizando a web (Selenium)

## 7.1 Introdução do capítulo

Selenium é uma ferramenta poderosa para controlar navegadores (Chrome, Firefox, Edge) como se fosse uma pessoa. É ótimo para tarefas como:

- Preencher formulários
- Coletar dados de sites (web scraping)
- Fazer login automático
- Testar sites

## 7.2 Explicação teórica simples

O Selenium "conversa" com o navegador através de um `webdriver`. Você dá comandos em Python e o navegador executa.

### O que você precisa:

- **Python**
- **Biblioteca Selenium** (`pip install selenium`)
- **WebDriver** (ex: ChromeDriver para Chrome, GeckoDriver para Firefox). Você baixa e coloca na pasta do seu projeto ou no PATH do sistema.

### Atenção

- Sites podem mudar, quebrando seu script.
- Respeite os termos de uso dos sites.
- Evite sobrecarregar servidores com muitas requisições.

## 7.3 Exemplo de código (abrir um site e pegar o título)

```python
from selenium import webdriver
import time

driver = webdriver.Chrome()  # Abre o navegador Chrome
driver.get("https://www.google.com")  # Navega para o Google
time.sleep(2)  # Espera 2 segundos para a página carregar

titulo = driver.title  # Pega o título da página
print("Título da página:", titulo)

driver.quit()  # Fecha o navegador
```

## 7.4 Explicação do código

- `webdriver.Chrome()` inicia o navegador.

- `driver.get(...)` abre a URL.
- `driver.title` pega o título da aba.
- `driver.quit()` fecha o navegador.

## 7.5 Exercício prático

Abra seu site favorito e imprima o título.

## 7.6 Desafio extra

Abra um site, procure por um elemento (ex: um parágrafo) e imprima o texto dele (dica: `driver.find_element_by_tag_name("p").text`).

---

# 8) Automatizando planilhas (Pandas)

## 8.1 Introdução do capítulo

Pandas é a biblioteca mais usada em Python para trabalhar com dados tabulares (planilhas, CSVs, bancos de dados). É como um Excel superpoderoso programável.

## 8.2 Explicação teórica simples

Com Pandas, você pode:

- Ler/Escrever CSV, Excel, JSON, etc.
- Filtrar dados
- Criar colunas
- Agrupar e somar
- Exportar um novo arquivo

## 8.3 Exemplo de código (relatório de vendas)

Suponha um arquivo `vendas.csv` com colunas: `produto`, `quantidade`, `preco`.

```
import pandas as pd

df = pd.read_csv("vendas.csv")

df["total"] = df["quantidade"] * df["preco"]

resumo = (
    df.groupby("produto", as_index=False)["total"]
    .sum()
    .sort_values("total", ascending=False)
)

resumo.to_csv("relatorio_por_produto.csv", index=False)

print("Relatório gerado: relatorio_por_produto.csv")
```

## 8.4 Explicação do código

- `pd.read_csv(...)` lê o CSV.
- `df["total"] = ...` cria a coluna `total`.
- `groupby("produto")` agrupa por produto.
- `["total"].sum()` soma o total por produto.
- `sort_values(...)` ordena do maior para o menor.
- `to_csv(...)` salva o relatório.

## 8.5 Exercício prático

Crie um `vendas.csv` com 10 linhas (pode fazer no Excel e salvar como CSV) e rode o script.

## 8.6 Desafio extra

Gere também um relatório por “faixa de preço” (ex: barato, médio, caro) criando uma coluna com regras.

---

# 9) Criando bots simples (juntando tudo)

## 9.1 Introdução do capítulo

Um “bot” simples é um programa que segue um fluxo e entrega um resultado com pouca intervenção humana.

## 9.2 Explicação teórica simples

Um bot geralmente tem:

- Entrada (configuração, arquivos, dados)
- Passos (processamento)
- Saída (relatório, print, arquivo, mensagem)

A base é: quebrar o problema em **pequenas funções**.

## 9.3 Exemplo de código (bot de checklist diário)

Esse bot cria um arquivo de checklist com data do dia.

```python
from datetime import datetime
from pathlib import Path

def data_hoje():
    return datetime.now().strftime("%Y-%m-%d")

def criar_checklist():
    pasta = Path("checklists")
    pasta.mkdir(exist_ok=True)

    arquivo = pasta / f"checklist_{data_hoje()}.txt"

    conteudo = (
        "Checklist do dia\n"
        "- [ ] Ver e-mails\n"
        "- [ ] Atualizar planilha\n"
        "- [ ] Revisar tarefas\n"
        "- [ ] Enviar relatório\n"
    )

    arquivo.write_text(conteudo, encoding="utf-8")
    return arquivo

arquivo_gerado = criar_checklist()
print("Checklist criado em:", arquivo_gerado)
```

## 9.4 Explicação do código

- `datetime.now()` pega data/hora atual.
- `strftime("%Y-%m-%d")` formata a data.
- Criamos a pasta `checklists`.
- Montamos o nome do arquivo com a data.
- `write_text(...)` salva o checklist.

## 9.5 Exercício prático

Adicione mais 3 itens no checklist e rode novamente.

## 9.6 Desafio extra

Faça o bot perguntar (via `input`) quais itens colocar no checklist e gerar com base na resposta.

---

# 10) Ideias para ganhar dinheiro com automação

## 10.1 Introdução do capítulo

Automação pode virar renda porque economiza tempo e reduz erros. Se algo gera valor para alguém, pode virar um serviço ou produto.

## 10.2 Explicação teórica simples

Formas comuns de monetizar:

- **Freelance**: automações sob medida (para uma empresa específica)
- **Consultoria**: melhorar processos, sugerir automações e implementar
- **Produtos simples**: scripts prontos + manual (templates)
- **Treinamento**: ensinar equipes a automatizar

### Onde automação dá mais retorno?

- Onde existe repetição
- Onde erros são caros
- Onde há muitos dados (planilhas, cadastros, relatórios)

**Regra de ouro**: antes de automatizar, descreva a tarefa como um checklist. Se você consegue explicar o passo a passo, você consegue transformar em código.

## 10.3 Exemplo de “proposta” (mini projeto)

“Vou automatizar a geração do relatório semanal: ler 3 planilhas, consolidar, gerar um CSV final e um PDF com gráficos.”

Mesmo que você não faça o PDF no começo, pode vender a automação em etapas:

1. Consolidar (Pandas)
2. Formatar (Excel/Pandas)
3. Enviar por e-mail (SMTP)
4. Agendar execução (Tarefa agendada / cron)

## 10.4 Explicação (como pensar no valor)

- Quanto tempo a pessoa perde por semana?
- Quanto custa 1 hora dessa pessoa?
- Quanto erro acontece manualmente?
- Quanto custa corrigir erros?

## 10.5 Exercício prático

Escolha uma tarefa do seu dia a dia e responda:

- O que é repetitivo?
- Qual entrada? Qual saída?
- Quais passos?

Escreva um checklist com 10 passos.

## 10.6 Desafio extra

Transforme seu checklist em um “pseudo-código” (quase código), por exemplo:

- “ler arquivo”, “filtrar dados”, “gerar relatório”

# Capítulo bônus: 10 ideias de projetos de automação que iniciantes podem criar

1. **Organizador de Downloads**: mover arquivos por extensão e data.
2. **Renomeador de fotos**: padronizar nomes com data e sequência.
3. **Gerador de relatórios**: ler CSV e gerar resumo por categoria (Pandas).
4. **Bot de checklist diário**: criar checklist com itens e data.
5. **Preenchedor de formulários**: abrir um site e preencher campos (Selenium).
6. **Coletor de preços**: coletar preços de um site e salvar no CSV (Selenium).
7. **Automação de mensagens**: gerar mensagens personalizadas a partir de planilha.
8. **Limpador de pastas**: mover arquivos antigos para `arquivo_morto/`.
9. **Backup automático**: copiar pastas importantes para outra pasta/drive.
10. **Bot "lembrete"**: ler tarefas de um arquivo e mostrar lembretes no horário (agendamento).

## Bônus 1 — Organizador de Downloads

### Introdução

Essa automação pega arquivos de uma pasta (ex.: Downloads) e organiza em subpastas por **extensão** (pdf, jpg, etc.) e/ou por **mês** (2026-04).

## Exemplo de código

```python
from pathlib import Path
import shutil
from datetime import datetime

pasta = Path("Downloads")  # troque para o caminho certo no seu computador

for arquivo in pasta.iterdir():
    if arquivo.is_file():
        extensao = arquivo.suffix.lower().replace(".", "")
        if extensao == "":
            extensao = "sem_extensao"

        data = datetime.fromtimestamp(arquivo.stat().st_mtime)
        mes = data.strftime("%Y-%m")

        destino = pasta / mes / extensao
        destino.mkdir(parents=True, exist_ok=True)

        shutil.move(str(arquivo), str(destino / arquivo.name))

print("Downloads organizados!")
```

## Exemplo de código (comentado para iniciantes)

```python
# Biblioteca para trabalhar com caminhos de arquivos/pastas de forma simples
from pathlib import Path

# Biblioteca que ajuda a mover/copiar arquivos
import shutil

# Biblioteca para trabalhar com datas
from datetime import datetime

# Escolha a pasta que você quer organizar
# Dica: se "Downloads" não for a pasta correta, troque pelo caminho completo
pasta = Path("Downloads")

# Percorre tudo que está dentro da pasta
for arquivo in pasta.iterdir():
    # Se for um arquivo (e não uma pasta), vamos tratar
    if arquivo.is_file():
        # Pega a extensão (por exemplo: ".pdf") e deixa tudo minúsculo
        extensao = arquivo.suffix.lower().replace(".", "")

        # Se não tiver extensão, colocamos um nome padrão
        if extensao == "":
            extensao = "sem_extensao"

        # Pega a data de modificação do arquivo (timestamp -> datetime)
        data = datetime.fromtimestamp(arquivo.stat().st_mtime)

        # Cria uma pasta por mês no formato "YYYY-MM"
        mes = data.strftime("%Y-%m")

        # Monta o destino final: Downloads/2026-04/pdf/ (exemplo)
        destino = pasta / mes / extensao

        # Cria as pastas necessárias
        destino.mkdir(parents=True, exist_ok=True)

        # Move o arquivo para o destino
        shutil.move(str(arquivo), str(destino / arquivo.name))

# Mensagem final para você saber que terminou
print("Downloads organizados!")
```

### Explicação do código (linha por linha)

- `Path("Downloads")` aponta para a pasta que você quer organizar (pode ser um caminho completo também).
- `pasta.iterdir()` lista tudo dentro da pasta.
- `arquivo.is_file()` garante que vamos mexer só com arquivos.
- `arquivo.suffix` pega a extensão (ex.: `.pdf`).
- `arquivo.stat().st_mtime` pega a data da última modificação do arquivo.
- `strftime("%Y-%m")` transforma a data em "ano-mês", como `2026-04`.
- `destino.mkdir(parents=True, exist_ok=True)` cria as pastas (inclusive as "do meio").
- `shutil.move(...)` move o arquivo para a pasta final.

### Exercício prático

Crie uma pasta `Downloads_teste`, coloque arquivos com extensões diferentes e rode o script apontando para essa pasta.

### Desafio extra

Faça o script ignorar arquivos menores que 5 KB (dica: `arquivo.stat().st_size`).

---

## Bônus 2 — Renomeador de fotos

### Introdução

Padronizar nomes de fotos ajuda muito a organizar. A ideia é renomear imagens para algo como `viagem_2026-04-26_001.jpg`.

## Exemplo de código

```
from pathlib import Path

pasta = Path("fotos")
prefixo = "viagem"

extensoes = [".jpg", ".jpeg", ".png"]
fotos = [p for p in pasta.iterdir() if p.is_file() and p.suffix.lower() in
extensoes]
fotos.sort()

for i, foto in enumerate(fotos, start=1):
    novo_nome = f"{prefixo}_{i:03d}{foto.suffix.lower()}"
    foto.rename(pasta / novo_nome)

print("Fotos renomeadas!")
```

## Exemplo de código (comentado para iniciantes)

```
from pathlib import Path  # trabalhar com pastas e arquivos

# Pasta onde estão as fotos
pasta = Path("fotos")

# Texto que vai aparecer no começo do nome do arquivo
prefixo = "viagem"

# Quais extensões vamos considerar como "foto"
extensoes = [".jpg", ".jpeg", ".png"]

# Lista só os arquivos que são imagem (tem extensão na lista)
fotos = [p for p in pasta.iterdir() if p.is_file() and p.suffix.lower() in
extensoes]

# Organiza para renomear em ordem
fotos.sort()

# enumerate cria um contador: 1, 2, 3...
for i, foto in enumerate(fotos, start=1):
    # i:03d cria 001, 002, 003...
    novo_nome = f"{prefixo}_{i:03d}{foto.suffix.lower()}"

    # Renomeia o arquivo (mantém na mesma pasta)
    foto.rename(pasta / novo_nome)

print("Fotos renomeadas!")
```

## Explicação do código

- `extensoes = [...]` define quais formatos vamos renomear.
- A lista `fotos` filtra apenas arquivos de imagem.
- `fotos.sort()` deixa em ordem (para o contador ficar previsível).
- `i:03d` cria números com 3 dígitos (001, 002, 003…).
- `rename(...)` renomeia/move o arquivo.

### Exercício prático

Crie 5 imagens (pode ser qualquer arquivo com extensão `.jpg` para testar) e rode.

### Desafio extra

Inclua a data no nome usando `datetime.now().strftime("%Y-%m-%d")`.

---

## Bônus 3 — Gerador de relatórios (Pandas)

### Introdução

Aqui você lê um CSV, faz contas (somar, agrupar, filtrar) e salva um relatório pronto para enviar.

### Exemplo de código

```
import pandas as pd

df = pd.read_csv("dados.csv")

df["Total"] = df["Quantidade"] * df["Preco"]

relatorio = (
    df.groupby("Categoria", as_index=False)["Total"]
    .sum()
    .sort_values("Total", ascending=False)
)

relatorio.to_excel("relatorio.xlsx", index=False)
print("Relatório gerado: relatorio.xlsx")
```

## Exemplo de código (comentado para iniciantes)

```
# Importa o Pandas (biblioteca de planilhas/dados)
import pandas as pd

# Lê o arquivo CSV e cria uma "tabela" (DataFrame)
df = pd.read_csv("dados.csv")

# Cria uma nova coluna "Total" multiplicando Quantidade * Preco
df["Total"] = df["Quantidade"] * df["Preco"]

# Agrupa por categoria e soma o total de cada categoria
relatorio = (
    df.groupby("Categoria", as_index=False)["Total"]  # agrupa por
"Categoria"
    .sum()                                            # soma os valores de
"Total"
    .sort_values("Total", ascending=False)             # ordena do maior
para o menor
)

# Salva em Excel (vai criar o arquivo se não existir)
relatorio.to_excel("relatorio.xlsx", index=False)

print("Relatório gerado: relatorio.xlsx")
```

## Explicação do código

- `read_csv` lê os dados.
- `df["Total"] = ...` cria uma coluna calculada.
- `groupby("Categoria")` agrupa por categoria.
- `sum()` soma os valores de cada grupo.
- `to_excel(...)` exporta para Excel.

## Exercício prático

Crie um `dados.csv` com colunas `Categoria`, `Quantidade`, `Preco` e gere o Excel.

### Desafio extra

Crie um relatório só com linhas onde `Quantidade` > 3 (dica: `df[df["Quantidade"] > 3]`).

---

## Bônus 4 — Bot de checklist diário

### Introdução

Um bot simples que cria um arquivo com tarefas para o dia. Ótimo para rotina.

### Exemplo de código

```
from datetime import datetime
from pathlib import Path

itens = ["Ver e-mails", "Revisar agenda", "Atualizar planilha", "Enviar
relatório"]

pasta = Path("checklists")
pasta.mkdir(exist_ok=True)

hoje = datetime.now().strftime("%Y-%m-%d")
arquivo = pasta / f"checklist_{hoje}.md"

linhas = ["# Checklist do dia", ""]
for item in itens:
    linhas.append(f"- [ ] {item}")

arquivo.write_text("\n".join(linhas), encoding="utf-8")
print("Checklist criado:", arquivo)
```

## Exemplo de código (comentado para iniciantes)

```python
from datetime import datetime  # para pegar a data de hoje
from pathlib import Path       # para lidar com pastas/arquivos

# Lista de tarefas (você pode trocar pelos seus itens)
itens = ["Ver e-mails", "Revisar agenda", "Atualizar planilha", "Enviar
relatório"]

# Cria/usa a pasta onde os checklists serão salvos
pasta = Path("checklists")
pasta.mkdir(exist_ok=True)

# Data no formato 2026-04-26
hoje = datetime.now().strftime("%Y-%m-%d")

# Nome do arquivo: checklist_2026-04-26.md
arquivo = pasta / f"checklist_{hoje}.md"

# Começa o conteúdo do arquivo
linhas = ["# Checklist do dia", ""]

# Para cada item, adiciona uma linha de checklist do Markdown
for item in itens:
    linhas.append(f"- [ ] {item}")

# Junta tudo com \n (quebra de linha) e salva em um arquivo de texto
arquivo.write_text("\n".join(linhas), encoding="utf-8")

print("Checklist criado:", arquivo)
```

## Explicação do código

- Criamos uma lista `itens` com tarefas.
- Geramos um nome de arquivo com a data.
- Montamos linhas no formato Markdown de checklist.
- Salvamos com `write_text`.

### Exercício prático

Adicione 5 itens seus (da sua rotina) e gere o checklist.

### Desafio extra

Peça os itens via `input` até a pessoa digitar `fim`.

---

## Bônus 5 — Preenchedor de formulários (Selenium)

### Introdução

Você abre um site e preenche campos automaticamente. Ideal para tarefas repetitivas (com permissão).

### Exemplo de código (modelo)

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("https://example.com/form")
time.sleep(2)

driver.find_element(By.NAME, "nome").send_keys("Gui")
driver.find_element(By.NAME, "email").send_keys("gui13python@gmail.com")

# driver.find_element(By.CSS_SELECTOR, "button[type=\'submit\']").click()

time.sleep(2)
driver.quit()
```

## Exemplo de código (comentado para iniciantes)

```
from selenium import webdriver              # controla o navegador
from selenium.webdriver.common.by import By # ajuda a localizar elementos na página
import time                                 # para pausas

# Abre o Chrome (precisa do navegador instalado)
driver = webdriver.Chrome()

# Abre a página do formulário
driver.get("https://example.com/form")

# Espera carregar (funciona, mas depois você pode usar esperas melhores)
time.sleep(2)

# Preenche o campo "nome" (ajuste o NAME conforme o site)
driver.find_element(By.NAME, "nome").send_keys("Gui")

# Preenche o campo "email"
driver.find_element(By.NAME, "email").send_keys("gui13python@gmail.com")

# Para sua segurança, o clique do enviar está comentado
# driver.find_element(By.CSS_SELECTOR, "button[type=\'submit\']").click()

# Espera só para você ver o resultado
time.sleep(2)

# Fecha o navegador
driver.quit()
```

## Explicação do código

- `get(...)` abre o site.
- `find_element(...)` encontra campos pelo seletor.
- `send_keys(...)` digita.
- O clique do submit ficou comentado para você testar com segurança.

### Exercício prático

Troque os seletores para um formulário de teste que você consiga inspecionar no navegador.

### Desafio extra

Use “esperas” (WebDriverWait) em vez de `time.sleep`.

---

## Bônus 6 — Coletor de preços (Selenium + CSV)

### Introdução

Coletar preços de um site e salvar em CSV pode virar um “comparador” simples.

## Exemplo de código (estrutura)

```python
import csv
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("https://example.com/produtos")
time.sleep(2)

cards = driver.find_elements(By.CSS_SELECTOR, ".produto")
dados = []

for c in cards:
    nome = c.find_element(By.CSS_SELECTOR, ".nome").text
    preco = c.find_element(By.CSS_SELECTOR, ".preco").text
    dados.append([nome, preco])

driver.quit()

with open("precos.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["nome", "preco"])
    writer.writerows(dados)
```

## Exemplo de código (comentado para iniciantes)

```
import csv                                # para salvar arquivo CSV
from selenium import webdriver            # para controlar o navegador
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("https://example.com/produtos")
time.sleep(2)  # espera carregar

# Pega uma lista de "cards" (cada produto)
cards = driver.find_elements(By.CSS_SELECTOR, ".produto")

# Aqui vamos guardar os dados coletados
dados = []

for c in cards:
    # Dentro do card, pega o texto do nome e do preço
    nome = c.find_element(By.CSS_SELECTOR, ".nome").text
    preco = c.find_element(By.CSS_SELECTOR, ".preco").text

    # Adiciona uma linha [nome, preco]
    dados.append([nome, preco])

driver.quit()  # fecha o navegador

# Salva em precos.csv
with open("precos.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["nome", "preco"])   # cabeçalho
    writer.writerows(dados)              # linhas

print("CSV gerado: precos.csv")
```

## Explicação do código

- `find_elements` pega vários itens.
- Para cada card, extraímos texto de nome e preço.
- `csv.writer` salva tudo em `precos.csv`.

### Exercício prático

Escolha um site de teste (ou uma página sua) e ajuste os seletores CSS.

### Desafio extra

Normalize o preço para número (ex.: “R$ 1.234,56” → `1234.56`).

---

## Bônus 7 — Automação de mensagens (Pandas + template)

### Introdução

You have provided a comprehensive content for the ebook. I will now proceed to create the Markdown file with the provided content, ensuring proper formatting for an ebook. After that, I will convert it to a PDF. This will complete phase 2 of our plan. You have a spreadsheet with names and data. The bot generates personalized messages for each person.

### Exemplo de código

```
import pandas as pd

df = pd.read_csv("contatos.csv")  # colunas: nome, produto, vencimento

template = "Olá, {nome}! Seu {produto} vence em {vencimento}. Posso ajudar?"

mensagens = []
for _, linha in df.iterrows():
    mensagens.append(template.format(**linha.to_dict()))

with open("mensagens.txt", "w", encoding="utf-8") as f:
    f.write("\n\n".join(mensagens))

print("Mensagens geradas em mensagens.txt")
```

## Exemplo de código (comentado para iniciantes)

```python
import pandas as pd  # para ler a planilha (CSV)

# Lê o arquivo CSV com seus contatos
# Ele precisa ter colunas: nome, produto, vencimento
df = pd.read_csv("contatos.csv")

# Template (modelo) de mensagem
template = "Olá, {nome}! Seu {produto} vence em {vencimento}. Posso ajudar?"

# Vamos guardar as mensagens aqui
mensagens = []

# Percorre cada linha da tabela
for _, linha in df.iterrows():
    # Transforma a linha em dicionário e preenche o template
    mensagens.append(template.format(**linha.to_dict()))

# Salva todas as mensagens em um arquivo de texto
with open("mensagens.txt", "w", encoding="utf-8") as f:
    f.write("\n\n".join(mensagens))

print("Mensagens geradas em mensagens.txt")
```

## Explicação do código

- Lemos a planilha com `read_csv`.
- `template.format(...)` preenche `{nome}`, `{produto}` e `{vencimento}`.
- Salvamos tudo num arquivo.

## Exercício prático

Crie `contatos.csv` com 5 pessoas e gere as mensagens.

## Desafio extra

Gere um arquivo por pessoa, com nome no arquivo (ex.: `mensagem_Ana.txt`).

---

# Bônus 8 — Limpador de pastas (arquivar antigos)

## Introdução

Esse bot pega arquivos antigos e move para uma pasta `arquivo_morto`.

## Exemplo de código

```
from pathlib import Path
import shutil
from datetime import datetime, timedelta

pasta = Path("minha_pasta")
arquivo_morto = pasta / "arquivo_morto"
arquivo_morto.mkdir(exist_ok=True)

limite = datetime.now() - timedelta(days=30)

for arquivo in pasta.iterdir():
    if arquivo.is_file():
        modificado = datetime.fromtimestamp(arquivo.stat().st_mtime)
        if modificado < limite:
            shutil.move(str(arquivo), str(arquivo_morto / arquivo.name))

print("Limpeza concluída!")
```

## Exemplo de código (comentado para iniciantes)

```python
from pathlib import Path            # caminhos
import shutil                       # mover arquivos
from datetime import datetime, timedelta  # datas

pasta = Path("minha_pasta")  # troque pela pasta que quer limpar

# Pasta para onde vão os arquivos antigos
arquivo_morto = pasta / "arquivo_morto"
arquivo_morto.mkdir(exist_ok=True)

# Tudo que for mais velho que 30 dias será movido
limite = datetime.now() - timedelta(days=30)

for arquivo in pasta.iterdir():
    if arquivo.is_file():
        # Data de modificação do arquivo
        modificado = datetime.fromtimestamp(arquivo.stat().st_mtime)

        # Se for mais antigo que o limite, move
        if modificado < limite:
            shutil.move(str(arquivo), str(arquivo_morto / arquivo.name))

print("Limpeza concluída!")
```

## Explicação do código

- `timedelta(days=30)` define “mais velho que 30 dias”.
- Comparamos datas e movemos o arquivo para `arquivo_morto`.

## Exercício prático

Crie arquivos e altere a data limite para `1` dia para testar rápido.

## Desafio extra

Em vez de mover, compacte em `.zip` (dica: `shutil.make_archive`).

---

# Bônus 9 — Backup automático (cópia de pastas)

## Introdução

Um backup simples copia arquivos importantes para outra pasta (ou HD externo).

## Exemplo de código

```
from pathlib import Path
import shutil

origem = Path("meus_documentos")
destino = Path("backup_meus_documentos")

destino.mkdir(exist_ok=True)

for item in origem.rglob("*"):
    if item.is_file():
        alvo = destino / item.relative_to(origem)
        alvo.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(item, alvo)

print("Backup concluído!")
```

## Exemplo de código (comentado para iniciantes)

```
from pathlib import Path  # para navegar em pastas
import shutil             # para copiar arquivos

# Pasta original (o que você quer salvar)
origem = Path("meus_documentos")

# Pasta onde o backup vai ficar
destino = Path("backup_meus_documentos")

# Cria a pasta destino se não existir
destino.mkdir(exist_ok=True)

# rglob("*") percorre todos os arquivos e pastas dentro de origem
for item in origem.rglob("*"):
    if item.is_file():
        # Mantém a mesma estrutura de pastas no backup
        alvo = destino / item.relative_to(origem)

        # Cria a pasta do arquivo no destino (se precisar)
        alvo.parent.mkdir(parents=True, exist_ok=True)

        # Copia o arquivo preservando informações (data de modificação,
etc.)
        shutil.copy2(item, alvo)

print("Backup concluído!")
```

## Explicação do código

- `rglob("*")` percorre a pasta inteira (incluindo subpastas).
- `relative_to(origem)` mantém a mesma estrutura no destino.
- `copy2` copia preservando datas/metadados.

## Exercício prático

Faça backup de uma pasta pequena de teste primeiro.

### Desafio extra

Faça o script ignorar arquivos maiores que 200 MB.

---

## Bônus 10 — Bot “lembrete” (ler tarefas e avisar)

### Introdução

Esse bot lê tarefas de um arquivo e mostra lembretes. É o começo de um “assistente” simples.

### Exemplo de código

```
import time
from pathlib import Path

arquivo = Path("tarefas.txt")

while True:
    if arquivo.exists():
        tarefas = [t.strip() for t in arquivo.read_text(encoding="utf-
8").splitlines() if t.strip()]
        print("Lembrete! Tarefas:")
        for t in tarefas:
            print("-", t)
    else:
        print("Crie o arquivo tarefas.txt para eu ler suas tarefas.")

    print("Aguardando 60 segundos...\n")
    time.sleep(60)
```

### Exemplo de código (comentado para iniciantes)

```python
import time              # para esperar alguns segundos
from pathlib import Path  # para lidar com arquivo

# Arquivo onde você escreve suas tarefas, uma por linha
arquivo = Path("tarefas.txt")

# Loop infinito (o programa fica rodando)
while True:
    # Verifica se o arquivo existe
    if arquivo.exists():
        # Lê todas as linhas do arquivo
        linhas = arquivo.read_text(encoding="utf-8").splitlines()

        # Remove espaços e ignora linhas vazias
        tarefas = [t.strip() for t in linhas if t.strip()]

        print("Lembrete! Tarefas:")
        for t in tarefas:
            print("-", t)
    else:
        print("Crie o arquivo tarefas.txt para eu ler suas tarefas.")

    # Espera 60 segundos e repete
    print("Aguardando 60 segundos...\n")
    time.sleep(60)
```

### Explicação do código

- O `vistos` guarda os nomes dos arquivos que já apareceram, para não repetir.
- A cada 5 segundos (`time.sleep(5)`), o script olha de novo a pasta.
- Quando encontra um arquivo novo, imprime na tela (você pode trocar por "mover", "renomear", etc.).

## Projetos extras (bem úteis no dia a dia)

Agora que você já viu 10 ideias, aqui vão **mais temas práticos** para você sentir que está automatizando coisas "de verdade".

# Bônus 11 — Organizador de notas (juntar vários .txt em 1)

## Introdução

Se você tem várias anotações soltas, dá para juntar tudo em um único arquivo, criando um “arquivo mestre”.

## Exemplo de código (comentado)

```
from pathlib import Path

# Pasta onde estão suas notas .txt
pasta = Path("minhas_notas")

# Arquivo final que vamos criar
saida = Path("notas_compiladas.txt")

partes = []

# Pega todos os .txt e ordena
arquivos = sorted(pasta.glob("*.txt"))

for arq in arquivos:
    # Título para separar as notas
    partes.append(f"\n--- {arq.name} ---\n")

    # Lê o conteúdo e adiciona
    partes.append(arq.read_text(encoding="utf-8"))

# Junta tudo e salva no arquivo final
saida.write_text("".join(partes), encoding="utf-8")
print("Notas compiladas em:", saida)
```

## Exercício prático

Crie 3 notas `.txt` e junte em um arquivo final.

### Desafio extra

Salve também uma versão `.md` colocando `#` antes do nome de cada nota.

---

## Bônus 12 — Conversor de imagens (PNG → JPG) para economizar espaço

### Introdução

Muitas vezes imagens PNG ficam pesadas. Converter para JPG pode reduzir tamanho.

### Exemplo de código (comentado)

> *Você precisa instalar:* `pip install pillow`

```
from pathlib import Path
from PIL import Image

pasta = Path("imagens")
destino = pasta / "convertidas_jpg"
destino.mkdir(exist_ok=True)

for img_path in pasta.glob("*.png"):
    # Abre a imagem
    img = Image.open(img_path)

    # Cria nome novo com extensão .jpg
    novo_nome = img_path.stem + ".jpg"

    # Salva como JPG (quality controla qualidade/tamanho)
    img.convert("RGB").save(destino / novo_nome, quality=85)

print("Conversão concluída!")
```

### Exercício prático

Coloque 5 PNGs na pasta e converta.

### Desafio extra

Faça o script também redimensionar para largura máxima de 1200px.

---

## Bônus 13 — Gerador de PDF simples (juntar textos em um PDF)

### Introdução

You can transform a text/report into a PDF to send by email.

### Exemplo de código (comentado)

> *Instale:* `pip install fpdf2`

```
from fpdf import FPDF

texto = "Relatório do dia\n\n- Tarefa 1 concluída\n- Tarefa 2 em
andamento\n"

pdf = FPDF()
pdf.add_page()
pdf.set_font("Helvetica", size=12)

for linha in texto.splitlines():
    pdf.cell(0, 10, txt=linha, ln=True)

pdf.output("relatorio.pdf")
print("PDF gerado: relatorio.pdf")
```

### Exercício prático

Gere um PDF com seu checklist do dia.

### Desafio extra

Leia o texto de um arquivo `.txt` em vez de deixar fixo no código.

---

# Bônus 14 — Monitor de “pasta de entrada” (quando cair arquivo novo, fazer algo)

## Introdução

Muito útil para trabalho: sempre que um arquivo novo aparecer em uma pasta, você pode mover, renomear ou registrar.

## Exemplo de código (comentado)

```
import time
from pathlib import Path

pasta = Path("entrada")
pasta.mkdir(exist_ok=True)

vistos = set()

print("Monitorando a pasta \'entrada\'...")

while True:
    for arq in pasta.iterdir():
        if arq.is_file() and arq.name not in vistos:
            vistos.add(arq.name)
            print("Arquivo novo detectado:", arq.name)

    time.sleep(5)  # verifica a cada 5 segundos
```

## Exercício prático

Rode o script e arraste um arquivo para a pasta `entrada`.

## Desafio extra

Quando detectar arquivo novo, mova automaticamente para uma subpasta `processados`.

### Explicação do código (bem direta)

- O `vistos` guarda os nomes dos arquivos que já apareceram, para não repetir.
- A cada 5 segundos ( `time.sleep(5)` ), o script olha de novo a pasta.
- Quando encontra um arquivo novo, imprime na tela (você pode trocar por "mover", "renomear", etc.).

---

## Bônus 15 — Renomear boletos/contas automaticamente (padronizar)

### Introdução

Quando você baixa contas e boletos, os nomes podem vir bagunçados. Esse script renomeia todos os PDFs com um padrão tipo `2026-04_internet.pdf`.

## Exemplo de código (comentado)

```python
from pathlib import Path
import re

# Pasta onde estão os PDFs
pasta = Path("contas")

# Regras simples: se o nome do arquivo tiver uma palavra, vira um nome
padronizado
regras = {
    "internet": "internet",
    "energia": "energia",
    "agua": "agua",
    "cartao": "cartao",
}

mes = "2026-04"  # troque para o mês atual

for arquivo in pasta.glob("*.pdf"):
    nome_original = arquivo.stem.lower()  # nome sem extensão

    # Procura qual regra combina com o nome do arquivo
    for palavra, nome_padrao in regras.items():
        if palavra in nome_original:
            novo_nome = f"{mes}_{nome_padrao}.pdf"
            arquivo.rename(pasta / novo_nome)
            print("Renomeado:", arquivo.name, "->", novo_nome)
            break
```

## Exercício prático

Crie 4 PDFs de teste com nomes contendo “internet”, “energia”, etc. e rode.

## Desafio extra

Se já existir um arquivo com o mesmo nome, adicione `_2`, `_3` … no final.

---

# Bônus 16 — Lista de compras a partir da geladeira (texto → lista limpa)

## Introdução

You type items in any format and the script cleans, removes duplicates, and organizes them alphabetically.

## Exemplo de código (comentado)

```
from pathlib import Path

entrada = Path("compras_bruto.txt")
saida = Path("lista_compras.txt")

# Lê as linhas, remove espaços e transforma em minúsculas
itens = [l.strip().lower() for l in entrada.read_text(encoding="utf-
8").splitlines()]

# Remove linhas vazias
itens = [i for i in itens if i]

# Remove repetidos e ordena
itens_unicos = sorted(set(itens))

# Salva
saida.write_text("\n".join(itens_unicos), encoding="utf-8")
print("Lista criada em:", saida)
```

## Exercício prático

Crie `compras_bruto.txt` com itens repetidos e com espaços bagunçados.

## Desafio extra

Crie categorias simples: se item tiver “leite/queijo/iogurte” vai para “Laticínios”, etc.

---

# Bônus 17 — Conversor de CSV para Excel (um clique)

## Introdução

You receive a CSV and need to send it “in Excel”. This script quickly converts it.

## Exemplo de código (comentado)

> *Precisa do Pandas:* `pip install pandas openpyxl`

```
import pandas as pd

arquivo_csv = "arquivo.csv"
arquivo_xlsx = "arquivo.xlsx"

df = pd.read_csv(arquivo_csv)
df.to_excel(arquivo_xlsx, index=False)

print("Convertido para:", arquivo_xlsx)
```

## Exercício prático

Pegue qualquer CSV e converta.

## Desafio extra

Converta automaticamente todos os CSVs de uma pasta.

---

# Bônus 18 — Encurtar nomes de arquivos longos (limpeza de “bagunça”)

## Introdução

Às vezes os arquivos vêm com nomes gigantes. Aqui você reduz e remove caracteres estranhos.

### Exemplo de código (comentado)

```
from pathlib import Path
import re

pasta = Path("bagunca")

def limpar_nome(nome: str) -> str:
    # Troca espaços por underscore
    nome = nome.replace(" ", "_")
    # Remove caracteres que não são letra, número, underscore, hífen ou
ponto
    nome = re.sub(r"[^a-zA-Z0-9_\-\.]", "", nome)
    # Limita tamanho
    return nome[:40]

for arq in pasta.iterdir():
    if arq.is_file():
        novo = limpar_nome(arq.name)
        if novo != arq.name:
            arq.rename(pasta / novo)
            print("Renomeado:", arq.name, "->", novo)
```

### Exercício prático

Crie arquivos com nomes com espaços e símbolos e rode.

### Desafio extra

Antes de renomear, gere um arquivo `antes_depois.csv` com o “mapa” de alterações.

---

## Bônus 19 — Extrator de e-mails de um texto (para contatos)

### Introdução

You have a large text (copied from messages/files) and want to extract only the emails, without repetition.

### Exemplo de código (comentado)

```
from pathlib import Path
import re

texto = Path("texto.txt").read_text(encoding="utf-8")

# Regex simples para e-mail
emails = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}",
texto)

# Remove repetidos e ordena
emails = sorted(set(emails))

Path("emails.txt").write_text("\n".join(emails), encoding="utf-8")
print("E-mails salvos em emails.txt")
```

### Exercício prático

Crie `texto.txt` com 10 e-mails repetidos e extraia.

### Desafio extra

Gere também um CSV com colunas `email` e `origem` (nome do arquivo).

---

## Bônus 20 — “Acordar” você com um alarme no PC (som + aviso)

### Introdução

A simple and fun project: play a sound and show a warning. Useful as a focus reminder.

### Exemplo de código (comentado)

```python
import time

segundos = 10  # troque para 1500 (25 min) se quiser um Pomodoro

print("Timer iniciado...")
time.sleep(segundos)

print("Tempo acabou! Levanta, bebe água, alonga 1 minuto.")
```

### Exercício prático

Mude o tempo para 5 segundos e teste.

### Desafio extra

Faça o script pedir o tempo via `input` e repetir 4 ciclos (Pomodoro).

---

## Próximos passos

- Refaça os exercícios com suas próprias ideias.
- Comece com automações pequenas e vá aumentando.
- Sempre teste em ambiente seguro (pasta de testes, conta de teste, etc.).

---

# Bônus Extra: Robô de Produtividade Completo

## Introdução

Este capítulo bônus apresenta um robô de produtividade que integra várias das automações que você aprendeu. Ele vai:

1. **Organizar seus downloads** por tipo e data.
2. **Gerar um checklist diário** com base em suas tarefas.
3. **Fazer backup** de uma pasta importante.

É um exemplo prático de como combinar diferentes scripts para criar uma solução mais robusta e útil no seu dia a dia.

## Pré-requisitos

Certifique-se de ter as bibliotecas `pandas` e `fpdf2` instaladas, caso utilize as funcionalidades de relatório e PDF. Para este exemplo, focaremos nas funcionalidades de organização de arquivos e checklist.

# Exemplo de Código: Robô de Produtividade

```
from pathlib import Path
import shutil
from datetime import datetime, timedelta

# --- Configurações --- #
PASTA_DOWNLOADS = Path("Downloads_para_organizar") # Crie esta pasta para
testar
PASTA_BACKUP_ORIGEM = Path("Meus_Documentos_Importantes") # Crie esta pasta
com alguns arquivos
PASTA_BACKUP_DESTINO = Path("Backup_Diario")
PASTA_CHECKLISTS = Path("Meus_Checklists")

ITENS_CHECKLIST = [
    "Ver e-mails importantes",
    "Revisar agenda do dia",
    "Atualizar planilha de projetos",
    "Responder mensagens pendentes",
    "Fazer uma pausa de 15 minutos"
]

# --- Funções de Automação --- #

def organizar_downloads(pasta_origem: Path):
    print(f"\nOrganizando downloads em: {pasta_origem}")
    pasta_origem.mkdir(exist_ok=True)

    for arquivo in pasta_origem.iterdir():
        if arquivo.is_file():
            extensao = arquivo.suffix.lower().replace(".", "")
            if extensao == "":
                extensao = "sem_extensao"

            data_modificacao =
datetime.fromtimestamp(arquivo.stat().st_mtime)
            mes_ano = data_modificacao.strftime("%Y-%m")

            destino = pasta_origem / mes_ano / extensao
            destino.mkdir(parents=True, exist_ok=True)

            shutil.move(str(arquivo), str(destino / arquivo.name))
            print(f"  Movido: {arquivo.name} -> {destino / arquivo.name}")
    print("Organização de downloads concluída.")
```

```python
def criar_checklist_diario(pasta_destino: Path, itens: list[str]):
    print(f"\nCriando checklist diário em: {pasta_destino}")
    pasta_destino.mkdir(exist_ok=True)

    hoje = datetime.now().strftime("%Y-%m-%d")
    arquivo_checklist = pasta_destino / f"checklist_{hoje}.md"

    linhas = [f"# Checklist do Dia {hoje}", ""]
    for item in itens:
        linhas.append(f"- [ ] {item}")
    linhas.append("") # Adiciona uma linha vazia no final

    arquivo_checklist.write_text("\n".join(linhas), encoding="utf-8")
    print(f"  Checklist criado: {arquivo_checklist}")

def fazer_backup(pasta_origem: Path, pasta_destino: Path):
    print(f"\nIniciando backup de: {pasta_origem} para {pasta_destino}")
    pasta_destino.mkdir(exist_ok=True)

    for item in pasta_origem.rglob("*"):
        if item.is_file():
            alvo = pasta_destino / item.relative_to(pasta_origem)
            alvo.parent.mkdir(parents=True, exist_ok=True)
            shutil.copy2(item, alvo)
            print(f"  Copiado: {item.name}")
    print("Backup concluído.")

# --- Execução do Robô --- #
if __name__ == "__main__":
    print("Iniciando Robô de Produtividade...")

    # 1. Organizar Downloads
    organizar_downloads(PASTA_DOWNLOADS)

    # 2. Criar Checklist Diário
    criar_checklist_diario(PASTA_CHECKLISTS, ITENS_CHECKLIST)

    # 3. Fazer Backup
    fazer_backup(PASTA_BACKUP_ORIGEM, PASTA_BACKUP_DESTINO)

    print("\nRobô de Produtividade finalizado com sucesso!")
```

## Como Usar e Testar

1. **Crie as pastas de teste**: Antes de rodar, crie as pastas `Downloads_para_organizar` e `Meus_Documentos_Importantes` na mesma localização do seu script. Coloque alguns arquivos de diferentes tipos e datas na pasta `Downloads_para_organizar` e alguns documentos na pasta `Meus_Documentos_Importantes`.
2. **Salve o código**: Copie o código acima e salve-o como `robo_produtividade.py`.
3. **Execute**: Abra seu terminal na pasta onde salvou o arquivo e execute: `python robo_produtividade.py`.
4. **Verifique os resultados**: Observe as pastas `Downloads_para_organizar` (agora organizada), `Meus_Checklists` (com o checklist do dia) e `Backup_Diario` (com a cópia dos seus documentos).

## Desafios Extras

- **Agendamento**: Use ferramentas como `cron` (Linux/macOS) ou o Agendador de Tarefas (Windows) para rodar este robô automaticamente todos os dias pela manhã.
- **Notificações**: Adicione uma notificação visual ou sonora ao final da execução (dica: `pyautogui.alert()` ou reproduzir um som).
- **Integração com E-mail**: Modifique o robô para enviar o checklist diário por e-mail para você.

Este robô é um ponto de partida. Sinta-se à vontade para personalizá-lo e adicionar mais funcionalidades que se encaixem na sua rotina!